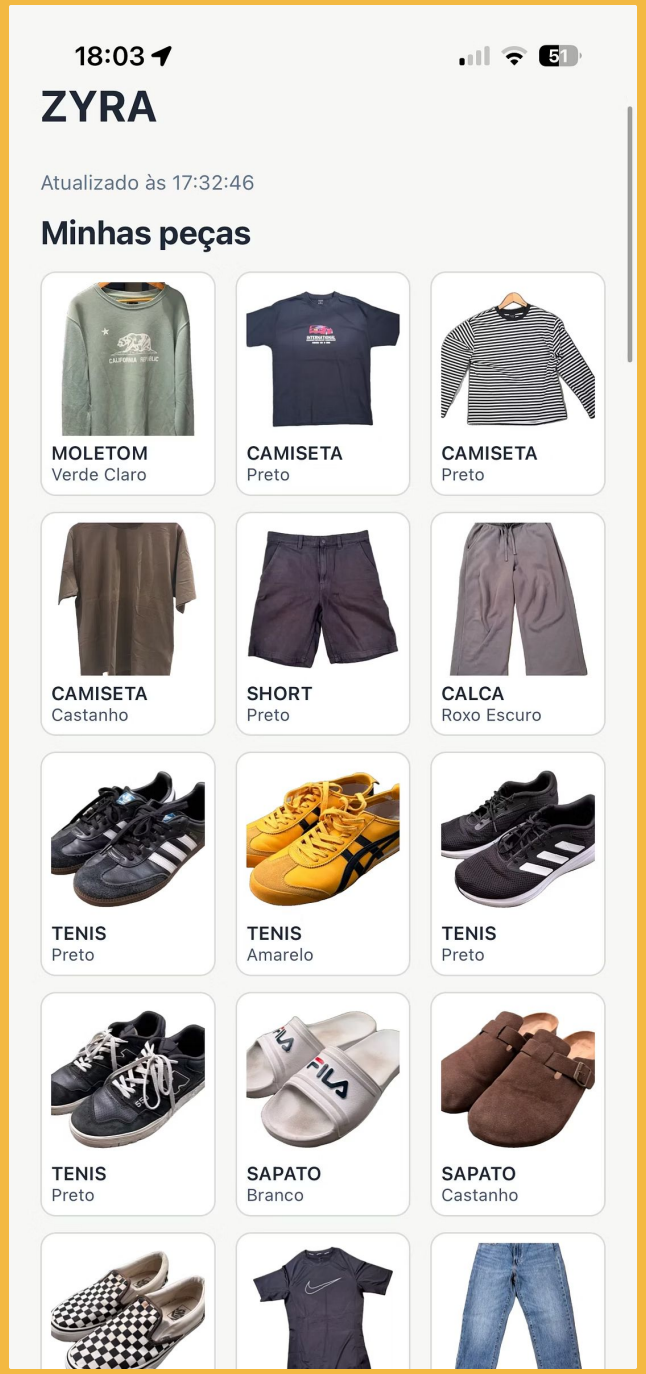
ZYRA

React Query na prova de conceito

Trabalho de Conclusão de Curso · Ciência da Computação · Prova de Conceito Mobile



Escopo da PoC

Sincronização de estado assíncrono no React Native com Expo Go, cobrindo rotas HTTP de looks e tolerância a falhas de rede.

Made with GAMMA

O ZYRA resolve um problema prático do guarda-roupa

identificação cromática e composição assistida por IA para daltonismo

- ***Acessibilidade visual***

Propósito principal. Aplicativo mobile desenhado para apoiar pessoas com daltonismo na identificação precisa de cores e na montagem autônoma de looks usando peças do próprio armário.

Chat inteligente

Sugestões direcionadas. Uma IA assistente propõe combinações harmônicas restritas exclusivamente ao acervo de roupas cadastradas, permitindo salvar ou descartar cada combinação gerada.

Desafio de sincronização

Resiliência de rede. Manter a listagem reativa e consistente após mutações (GET, POST e DELETE) sem exigir recargas manuais, preservando usabilidade quando a API falha.

Made with GAMMA

A pergunta é como atualizar a lista sem recarga manual

a questão central de arquitetura de dados na interface mobile

Problema de pesquisa da PoC

Como o @tanstack/react-query permite listar, salvar e apagar os looks do ZYRA (GET/POST/DELETE /looks) de forma que a lista se atualize sozinha depois de cada alteração e o app continue utilizável quando o back fica fora do ar?

Requisitos de resposta

Atualização reativa. Descarte do padrão de recarga por evento imperativo ou gatilhos dispersos de tela.

Degradação suave. Preservação do cache local em memória diante de timeout ou indisponibilidade da API.

Made with GAMMA

React Query foi a alternativa mais completa para a PoC

comparativo de abordagens para consumo e sincronização de API

Opção	*Serve para*	*Expo Go*	*Decisão*
fetch direto (apiRequest atual)	Requisição HTTP direta; estado de loading, tratamento de erro e recarga ficam totalmente a cargo de quem chama.	Sim	**Não:** recarga manual, ausência de cache unificado e sem mecanismo automático de retry.
hook próprio useFetch	Centralizar carregando, erro e dados de resposta sob um invólucro reaproveitável de componente.	Sim	**Não:** cache compartilhado, invalidação declarativa e lógica de retry ainda precisariam ser escritos à mão.
• ***@tanstack/react-query***	Gerenciamento completo: cache em memória, `useQuery`, `useMutation`, `invalidateQueries` e retry configurável.	Sim	• **Sim:** invalidação atualiza a lista após mutação e queries executam 3 tentativas automáticas.

Critérios avaliados no ambiente de prototipagem Expo SDK sem necessidade de módulos nativos adicionais.

Made with GAMMA

Mutação concluída, query invalidada, lista atualizada

o fluxo operacional de atualização e tolerância a quedas

App ZYRA + useQuery	useMutation	onSuccess	API /looks → cache
Busca peças e looks com GET /looks e mantém os dados em cache.	Executa salvar (POST) ou apagar (DELETE) em /looks, sem retries automáticos.	Chama invalidateQueries(['looks']); a query é buscada de novo.	GET, POST e DELETE /looks atualizam a lista no app.

Caminho de falha: se o back falhar, a query tenta novamente 3 vezes; a última lista continua visível no cache.

Made with GAMMA

Demonstração

Made with GAMMA

As IAs erraram quando afirmaram mais do que verificaram

análise crítica dos erros conceituais e alucinações durante a prova de conceito

ChatGPT

Alucinação e premissa falsa. Afirmou categoricamente ter verificado a documentação, mas citou link quebrado para a biblioteca Zustand.

Erro conceitual. Alertou sobre risco de repetição automática em mutations, ignorando que mutations não possuem retry habilitado por padrão no React Query.

Claude

Aglutinação indevida. Mesclou a implementação de `fetch` nativo com o hook customizado `useFetch` em um mesmo raciocínio.

Rastreabilidade fraca. Apontou link inacessível e fundamentou recomendações em fontes não oficiais para justificar a compatibilidade.

Copilot

Ferramental não acionado. Declarou ter consultado os repositórios do Expo sem acionar as ferramentas de checagem do ambiente.

Suposição de erros. Ignorou as convenções de código do projeto e inventou a mensagem exata de erro retornada pela falha de rede.

Made with GAMMA

A PoC passou, o TCC agora precisa transformar o aprendizado em produto

01 ***Expansão dos hooks nas telas***

Levar `useQuery`, `useMutation` e `invalidateQueries` para as telas de looks e peças.

02 ***QueryClient em arquivo separado***

Criar o QueryClient em arquivo separado para evitar problemas com Fast Refresh.

03 ***Token e cache sem conexão***

Renovar token após 401 e avaliar persistência do cache para uso sem conexão.

Limitações atuais

Cache: só em memória.

Token: expira em 1 hora.

IP do Mac: muda conforme a rede.

Made with GAMMA

Fontes

documentação técnica oficial e referências consultadas

React Native — **Networking Documentation**. Práticas e primitivas para consumo HTTP.

TanStack Query — **Guia de integração com React Native**.

TanStack Query — **Comportamento de mutações e ausência de retry por padrão**.

TanStack Query — **Network Mode e gestão de reconexão**.

TanStack Query — **Important Defaults: staleTime e cacheTime**.

Expo — **Using third-party libraries no ecossistema gerenciado**.

Expo SDK — **Módulo NetInfo para monitoramento de estado de rede**.

React Native Directory — **Compatibilidade de @tanstack/react-query com Expo Go**.

Todas as fontes referenciadas foram verificadas em novembro de 2026 para fins de validação da arquitetura técnica do TCC.

Made with GAMMA